

IT353 BLOCKCHAIN TECHNOLOGY



Dr. Shamama Anwar

Assistant Professor

Department of Computer Science and Engineering,

BIT, Mesra

shamama@bitmesra.ac.in

9431103880

Module - III

Mining and simulating blockchain:

- Game theory behind competitive mining.
- Incentives: mining and transaction fees.
- Energy expended in mining.
- Blockchain Simulation

Game theory

- Game theory is the study of strategic decision making.
- It attempts to determine mathematically and logically the actions that “players” should take to secure the best outcomes for themselves in a wide array of “games.”
- Provides tools for analyzing situations in which players make decisions that are interdependent.
- This interdependence causes each player to consider the other player’s possible decisions, or strategies, in formulating strategy.
- Game theory has been applied to a wide variety of situations in which the choices of players interact to affect the outcome.
- The Prisoner’s Dilemma
 - Two people are supposed to be arrested for a crime.
 - They have to serve 2-year imprisonment for this.
 - The police also suspects that these two people have also committed another bigger crime.
 - The police placed each prisoner in a separate cell. Both of them are told that they are suspects of being associated with a bigger crime. They are inquired separately and are not able to communicate with each other.
 - The prisoners are given two situations:
 - To confess or remain silent

Game theory

		Prisoner B	
		Confess	Remain Silent
Prisoner A	Confess	(2, 2)	(0,7)
	Remain Silent	(7,0)	(5,5)

- The system favors a specific outcome (1, 2 both confess).
- The prisoner's dilemma shows that if the prisoners pursue their own self-interest the result is sub-optimal as the best option is co-operation.
- But as the consequence of co-operation is higher, game theory tell us that a rational actor will opt otherwise.
- According to game theory there are three core elements of any game:
 - **Players:** The strategic actors within a game
 - **Strategy:** A plan of action a player will take given the circumstances that will arise
 - **Payoff:** The result or outcome a player receives after achieving a certain state.

Game theory

- Consensus algorithms such as Proof of Work rely on game theory in order to maintain trust-less co-operation.
- In a competitive mining environment where solving puzzles is resource intensive, game theory tells us that rational actors are incentivized to act honestly in order to not risk losing their investment.
- Bad faith actors are disincentivized to cheat — i.e., accepting invalid transactions or 'double spending' — as the consequence would typically involve a loss of resources.
- Mining can be viewed as a repeated prisoner's dilemma where each node employs a strategy that aims to maximize their payout without considering the strategy of other players.

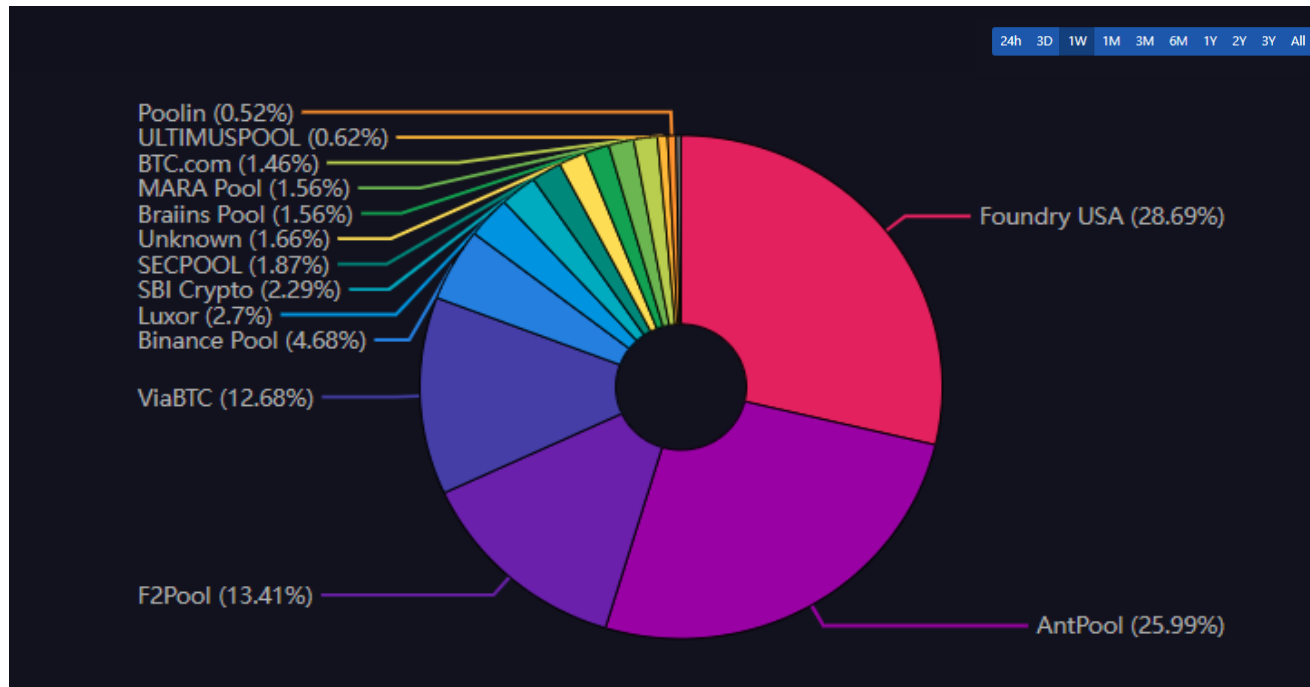
		Miner 2	
		Dishonest	Honest
Miner 1	Dishonest	(0,0)	(0,5)
	Honest	(5,0)	(5,5)

Game theory in blockchain

- A **mining pool** is a group of miners who connect their mining machines over a network to boost their chances of earning the reward for opening a new block.
- New blocks are opened when a miner discovers the solution to the problem the entire blockchain network is trying to solve. This process is time-consuming, energy-intensive, and requires a computer that can generate and check billions, or even better, trillions of randomized hexadecimal numbers per second.
- Individually, participants in a mining pool contribute their processing power toward the effort of finding a block. If the pool is successful in these efforts, they receive a reward, typically in the form of the associated cryptocurrency.
- Rewards are usually divided between the individuals who contributed, according to the proportion of each individual's processing power or work relative to the whole group, usually called shares.
- Solo mining involves working alone, while pool mining entails collaboration with other miners. In pool mining, participants combine their computing power to enhance the likelihood of mining a block.

Game theory in blockchain

- Why Mining pool: Risk sharing
- Bigger Pools (less risk) charge fees and compete amongst other pools for fee fixation
- Each pool charges optimal fee given other's fees (Game theory)
- Optimal allocation – Diversity !!



<https://mempool.space/graphs/mining/pools>

Incentive: Mining and Transaction fees

- **Transaction fees** are the expenses that entitled need to pay to their payment service provider every time the provider processes an electronic transaction.
- Transaction fees can vary slightly, depending on the payment service provider.
- Every cryptocurrency also has a transaction fees built into their basic operating structure. Bitcoin (BTC 0.98%) calls it a network fee, Ethereum (ETH 1.26%) calls it gas fee.
- Transaction fees serve two purposes in blockchain:
 - Reward miners or validators
 - Protect the network from spam attacks.
- The congestion in the Mempool significantly impacts transaction processing times and fees. With a growing number of transactions competing for validation, miners prioritized those with higher fees.
- The amount of the blockchain fee varies depending on the cryptocurrency, network congestion, and the size of the transaction.
- The fee is usually calculated based on the size of the transaction in bytes. The larger the transaction size, the higher the fee required to process the transaction. It's worth noting that blockchain fees can be quite high during times of high network congestion.

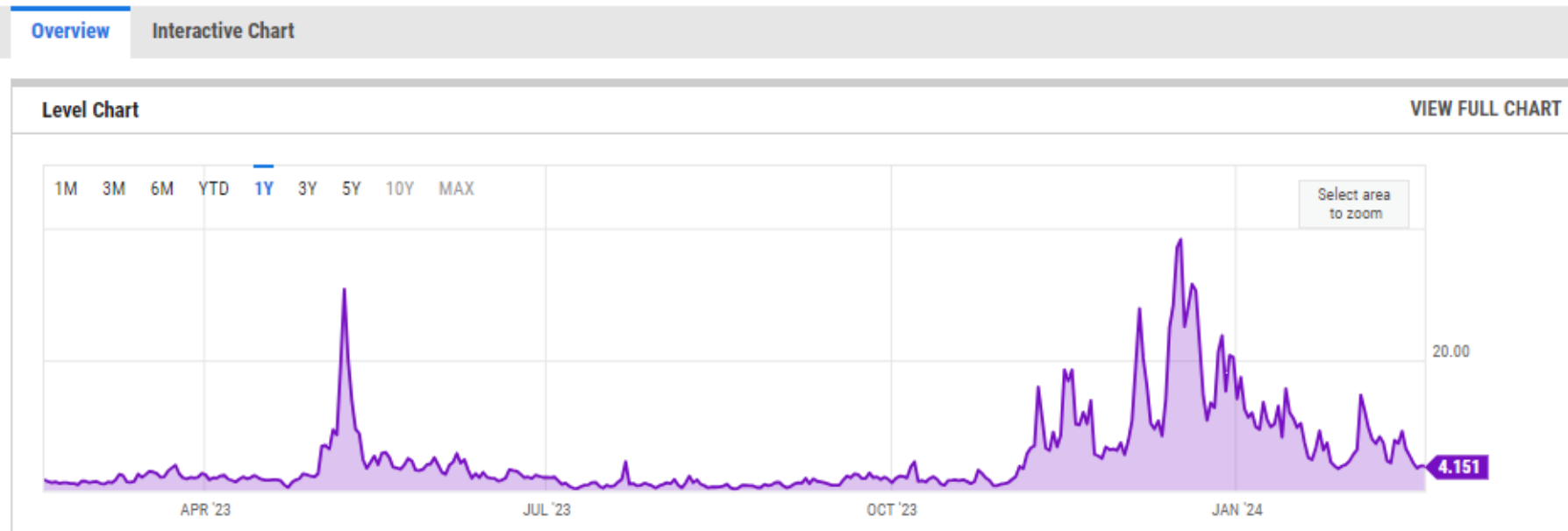
Incentive: Mining and Transaction fees

- Bitcoin

- Fees are not dependent on the amount sent but on the transaction size (in bytes).
- Eg: Transaction size: 400 bytes
- Transaction fee: 80 satoshis per byte.
- Amount payable: 32,000 satoshis (or 0.00032 BTC)

Bitcoin Average Transaction Fee (I:BATF)

4.151 USD/tx for Feb 20 2024



https://ycharts.com/indicators/bitcoin_average_transaction_fee

Incentive: Mining and Transaction fees

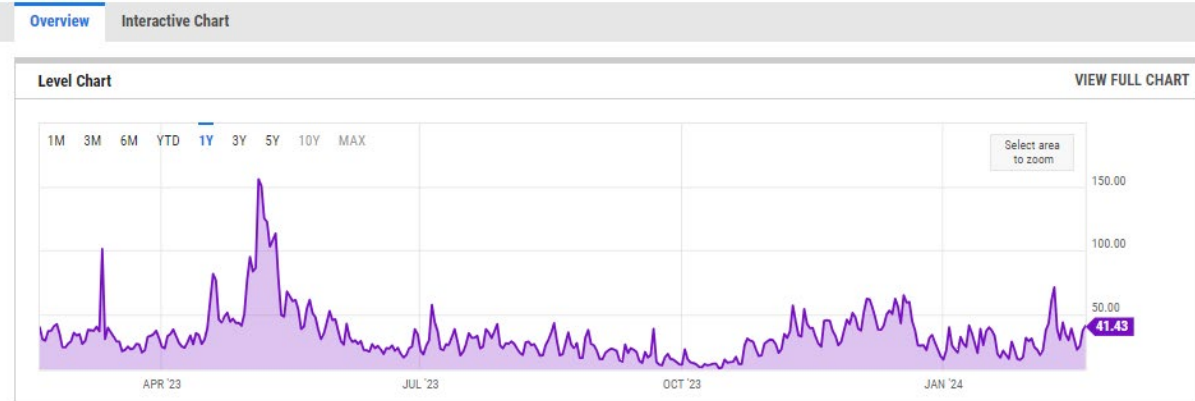
- Ethereum

- Ethereum transaction fees take into account the amount of computing power needed to process a transaction, known as gas.
- Gas also has a variable price measured in ether (ETH).
- While the gas needed for a specific transaction can stay the same, gas prices can rise or fall.
- Miners will likely prioritize transactions with high gas.
- Ethereum fees are determined by:
 - Complexity. Specifically, the amount of gas required to process a transaction.
 - Base fee. The reserve price below which a transaction is ineligible for processing on the blockchain.
 - Priority fee. An optional incentive fee to expedite processing time.
- Eg: Transaction cost: 21,000 gas
- Gas price: 71 Gwei.
- Total transaction fee: 1,491,000 Gwei or 0.001491 ETH.

Incentive: Mining and Transaction fees

Ethereum Average Gas Price (I:EGPND)

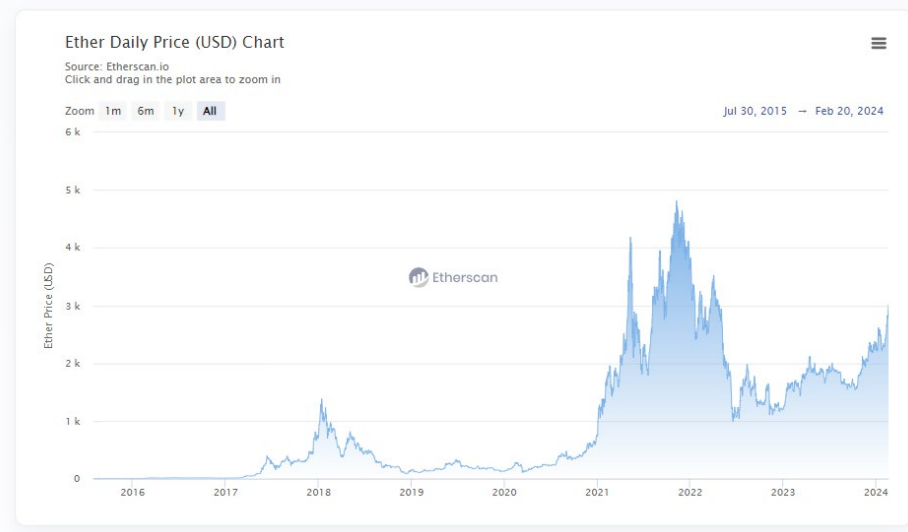
41.43 Gwei for Feb 20 2024



https://ycharts.com/indicators/ethereum_average_gas_price

Ether Daily Price (USD) Chart

Featured: Elevate your website with curated name tags and labels 🍌 in the all-new Metadata API. [Learn More.](#)



<https://etherscan.io/gastracker>

Energy expended in Mining

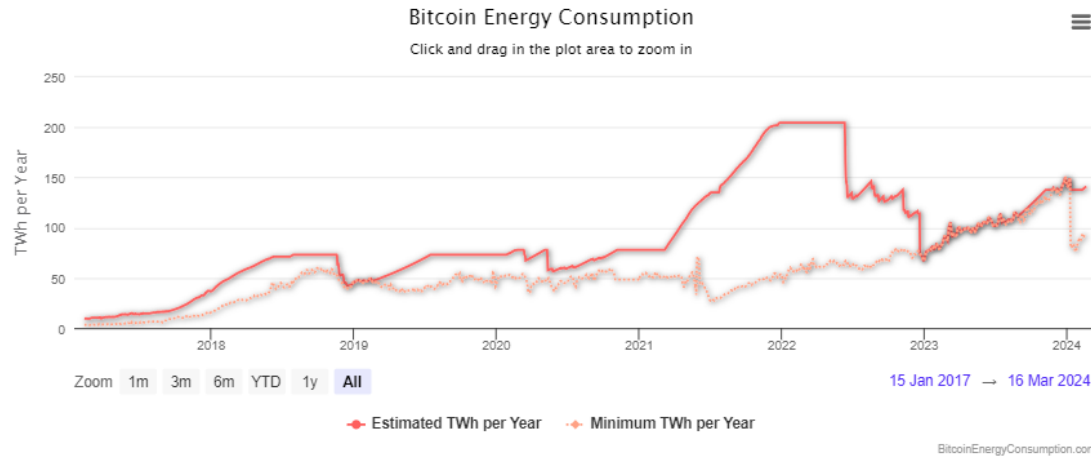
- When bitcoins are traded, computers across the globe race to complete a computation that creates a 64-digit hexadecimal number, or hash, for that bitcoin.
- This hash goes into a public ledger so anyone can confirm that the transaction for that particular bitcoin happened.
- The computer that solves the computation first gets a reward of 6.2 bitcoins, or about \$134,000 at current prices (as per 2023).
- Such powerful computers, termed as Rigs, usually use powerful GPUs to handle calculations and require high-wattage power supplies.
- The graphics cards on mining rigs work 24 hours a day and a rig with three GPUs can consume 1,000 watts of power or more when it's running (equivalent to a AC).
- Crypto mining businesses can have hundreds or even thousands of rigs in one location.
- Not only do rigs take up power, they also generate heat, hence a requirement for cooling facility.
- The Digiconomist's Bitcoin Energy Consumption Index estimated that one bitcoin transaction takes 1,449 kWh to complete, or the equivalent of approximately 50 days of power for the average US household.

Energy expended in Mining

Bitcoin Energy Consumption Index

The Bitcoin Energy Consumption Index provides the latest estimate of the total energy consumption of the Bitcoin network.

NEW RESEARCH: "Bitcoin's growing water footprint" (November 2023); A single Bitcoin transaction could cost as much water as a backyard swimming pool.



Single Bitcoin Transaction Footprints

Carbon Footprint

643.23 kgCO₂



Equivalent to the carbon footprint of 1,425,623 VISA transactions or 107,205 hours of watching Youtube.

Electrical Energy

1153.24 kWh



Equivalent to the power consumption of an average U.S. household over 39.53 days.

Electronic Waste

231.30 grams



Equivalent to the weight of 1.41 iPhones 12 or 0.47 iPads. (Find more info on e-waste [here](#).)

Fresh Water Consumption

18,175 liters



Equivalent to the amount of water in a backyard swimming pool.

Simulating a Blockchain

- Creating own blockchain:
 - Build one from scratch
 - Use a pre-build open source blockchain network Ethereum, Hyperledger, etc.
 - Fork, modify and/or improve an existing blockchain open-source code.

Build one from scratch

```
1 # Start
2 import datetime as d
3 import hashlib as h
4
5 class Block:
6     def __init__(self, index, timestamp, data, prevhash):
7         self.index = index
8         self.timestamp = timestamp
9         self.data = data
10        self.prevhash = prevhash
11        self.hash = self.hashblock()
12
13    def hashblock(self):
14        block_encryption = h.sha256()
15        block_encryption.update(bytes(str(self.index), 'utf-8'))
16        return block_encryption.hexdigest()
17
18    @staticmethod
19    def genesisblock():
20        return Block(index=0, timestamp=d.datetime.now(), data="genesis block transaction", prevhash=" ")
```

Simulating a Blockchain

```
21  @staticmethod
22  def newblock(lastblock):
23      index = lastblock.index+1
24      timestamp = d.datetime.now()
25      hashblock = lastblock.hash
26      data = "Transaction " +str(index)
27      return Block(index,timestamp,data,hashblock)
28
29
30  blockchain = [Block.genesisblock()]
31  prevblock = blockchain[0]
32
33  print("Block ID :{} ".format(prevblock.index) )
34  print("Timestamp:{}".format(prevblock.timestamp))
35  print("Hash of the block:{}".format(prevblock.hash))
36  print("Previous Block Hash:{}".format(prevblock.prevhash))
37  print("data:{}\n".format(prevblock.data))
38
39
40  for i in range (0,3):
41      addblock = Block.newblock(prevblock)
42      blockchain.append(addblock)
43      prevblock =addblock
44
45      print("Block ID #{} ".format(addblock.index))
46      print("Timestamp:{}".format(addblock.timestamp))
47      print("Hash of the block:{}".format(addblock.hash))
48      print("Previous Block Hash:{}".format(addblock.prevhash))
49      print("data:{}\n".format(addblock.data))
```

Simulating a Blockchain

```
"C:\Users\Shamama\PycharmProjects\BC project1\.venv\Scripts\python.exe" "C:\Users\Shamama\PycharmProjects\BC project1\main.py"
Block ID :0
Timestamp:2024-02-21 15:57:08.337972
Hash of the block:5feceb66ffc86f38d952786c6d696c79c2dbc239dd4e91b46729d73a27fb57e9
Previous Block Hash:
data:genesis block transaction

Block ID #1
Timestamp:2024-02-21 15:57:08.337972
Hash of the block:6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b
Previous Block Hash:5feceb66ffc86f38d952786c6d696c79c2dbc239dd4e91b46729d73a27fb57e9
data:Transaction 1

Block ID #2
Timestamp:2024-02-21 15:57:08.337972
Hash of the block:d4735e3a265e16eee03f59718b9b5d03019c07d8b6c51f90da3a666eec13ab35
Previous Block Hash:6b86b273ff34fce19d6b804eff5a3f5747ada4eaa22f1d49c01e52ddb7875b4b
data:Transaction 2

Block ID #3
Timestamp:2024-02-21 15:57:08.337972
Hash of the block:4e07408562bedb8b60ce05c1decfe3ad16b72230967de01f640b7e4729b49fce
Previous Block Hash:d4735e3a265e16eee03f59718b9b5d03019c07d8b6c51f90da3a666eec13ab35
data:Transaction 3
```


Simulating a Blockchain

Using a pre-built framework

- Node.js

```
C:\Windows\system32\cmd.e: X + v
Your environment has been set up for using Node.js 20.11.1 (ia32) and npm.
C:\Users\Shamama>npm i -g npm
added 1 package in 2s
C:\Users\Shamama>node --version
v20.11.1
```

- Truffle

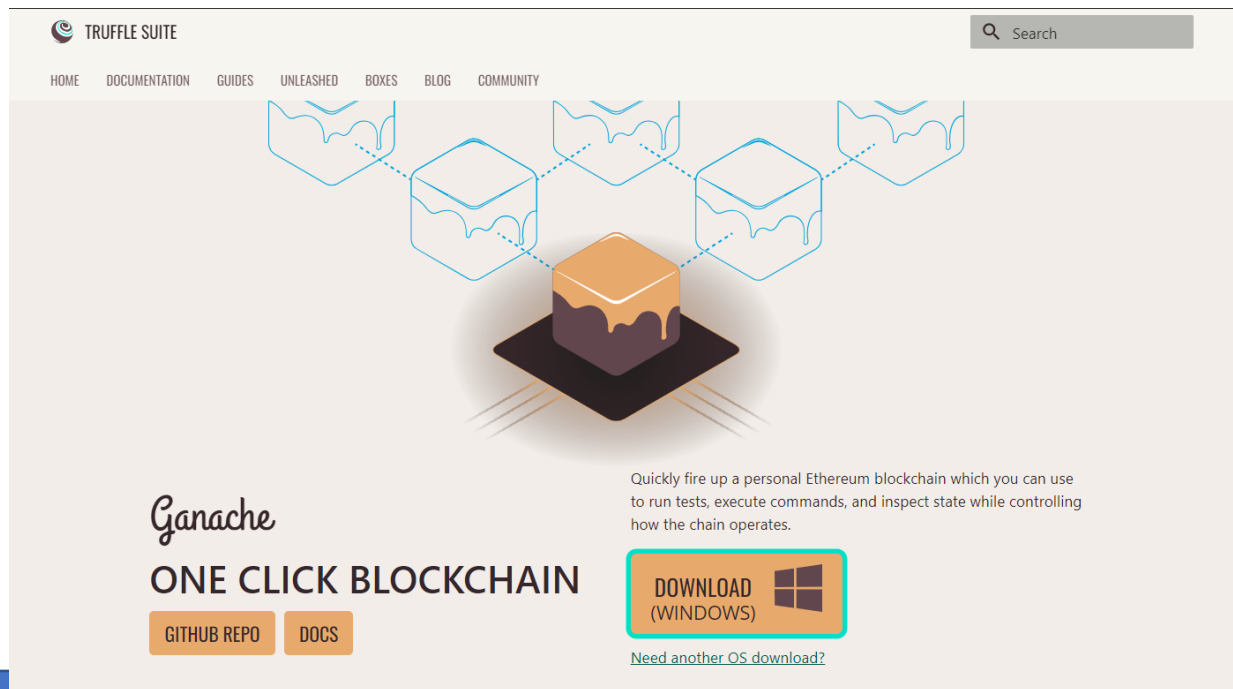
- Framework for building decentralized applications (Dapps) on the Ethereum blockchain.
- Smart contract creation and testing to deployment and asset management.
- The Truffle Suite's provides a user-friendly environment for developers to write, compile, and deploy smart contracts efficiently.
- Truffle also integrates with popular testing frameworks and allows developers to simulate real-world scenarios, ensuring the reliability and security of their smart contracts before deployment.

```
C:\Users\Shamama>npm install -g truffle
npm WARN deprecated har-validator@5.1.5: this library is no longer supported
npm WARN deprecated mkdirp-promise@5.0.1: This package is broken and no longer maintained. 'mkdirp' itself supports promises now, please switch to that.
npm WARN deprecated request@2.88.2: request has been deprecated, see https://github.com/request/request/issues/3142
npm WARN deprecated multibase@0.6.1: This module has been superseded by the multiformats module
npm WARN deprecated multicodec@0.5.7: This module has been superseded by the multiformats module
npm WARN deprecated cids@0.7.5: This module has been superseded by the multiformats module
npm WARN deprecated apollo-server-types@3.8.0: The 'apollo-server-types' package is part of Apollo Server v2 and v3, which are now deprecated (end-of-life October 22nd 2023 and October 22nd 2024, respectively). This package's functionality is now found in the '@apollo/server' package. See https://www.apollographql.com/docs/apollo-server/previous-versions/ for more details.
npm WARN deprecated apollo-server-plugin-base@3.7.2: The 'apollo-server-plugin-base' package is part of Apollo Server v2 and v3, which are now deprecated (end-of-life October 22nd 2023 and October 22nd 2024, respectively). This package's functionality is now found in the '@apollo/server' package. See https://www.apollographql.com/docs/apollo-server/previous-versions/ for more details.
npm WARN deprecated apollo-server-errors@3.3.1: The 'apollo-server-errors' package is part of Apollo Server v2 and v3, which are now deprecated (end-of-life October 22nd 2023 and October 22nd 2024, respectively). This package's functionality is now found in the '@apollo/server' package. See https://www.apollographql.com/docs/apollo-server/previous-versions/ for more details.
npm WARN deprecated apollo-datasource@3.3.2: The 'apollo-datasource' package is part of Apollo Server v2 and v3, which are now deprecated (end-of-life October 22nd 2023 and October 22nd 2024, respectively). See https://www.apollographql.com/docs/apollo-server/previous-versions/ for more details.
npm WARN deprecated apollo-server-env@4.2.1: The 'apollo-server-env' package is part of Apollo Server v2 and v3, which are now deprecated (end-of-life October 22nd 2023 and October 22nd 2024, respectively). This package's functionality is now found in the '@apollo/usage-reporting-protobuf' package. See https://www.apollographql.com/docs/apollo-server/previous-versions/ for more details.
npm WARN deprecated apollo-reporting-protobuf@3.4.0: The 'apollo-reporting-protobuf' package is part of Apollo Server v2 and v3, which are now deprecated (end-of-life October 22nd 2023 and October 22nd 2024, respectively). This package's functionality is now found in the '@apollo/usage-reporting-protobuf' package. See https://www.apollographql.com/docs/apollo-server/previous-versions/ for more details.
npm WARN deprecated testrpc@0.0.1: testrpc has been renamed to ganache-cli, please use this package from now on.
npm WARN deprecated @ensdomains/ens@0.4.5: Please use @ensdomains/ens-contracts
npm WARN deprecated @ensdomains/resolver@0.2.4: Please use @ensdomains/ens-contracts
npm WARN deprecated uuid@3.4.0: Please upgrade to version 7 or higher. Older versions may use Math.random() in certain circumstances, which is known to be problematic. See https://v8.dev/blog/math-random for details.
npm WARN deprecated multibase@0.7.0: This module has been superseded by the multiformats module
npm WARN deprecated multicodec@1.0.4: This module has been superseded by the multiformats module
npm WARN deprecated uuid@2.0.1: Please upgrade to version 7 or higher. Older versions may use Math.random() in certain circumstances, which is known to be problematic. See https://v8.dev/blog/math-random for details.
added 1153 packages in 3m
```

Simulating a Blockchain

- Ganache

- Ganache is a personal blockchain developed as part of the Truffle Suite.
- It provides a local Ethereum network for developers to test their dapps.
- Ganache allows developers to create and manage accounts, simulate transactions, and inspect blockchain activity, all within a user-friendly interface.
- It eliminates the need for connecting to the Ethereum mainnet or test networks during development, allowing for faster iterations and improved debugging capabilities.
- Ganache also offers advanced features like network snapshots and deterministic blockchain behavior, enabling developers to reproduce and test specific scenarios for their dapps precisely.



Simulating a Blockchain

Ganache

ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK
0

GAS PRICE
20000000000

GAS LIMIT
6721975

HARDFORK
MERGE

NETWORK ID
5777

RPC SERVER
HTTP://127.0.0.1:7545

MINING STATUS
AUTOMINING

WORKSPACE
QUICKSTART

SAVE

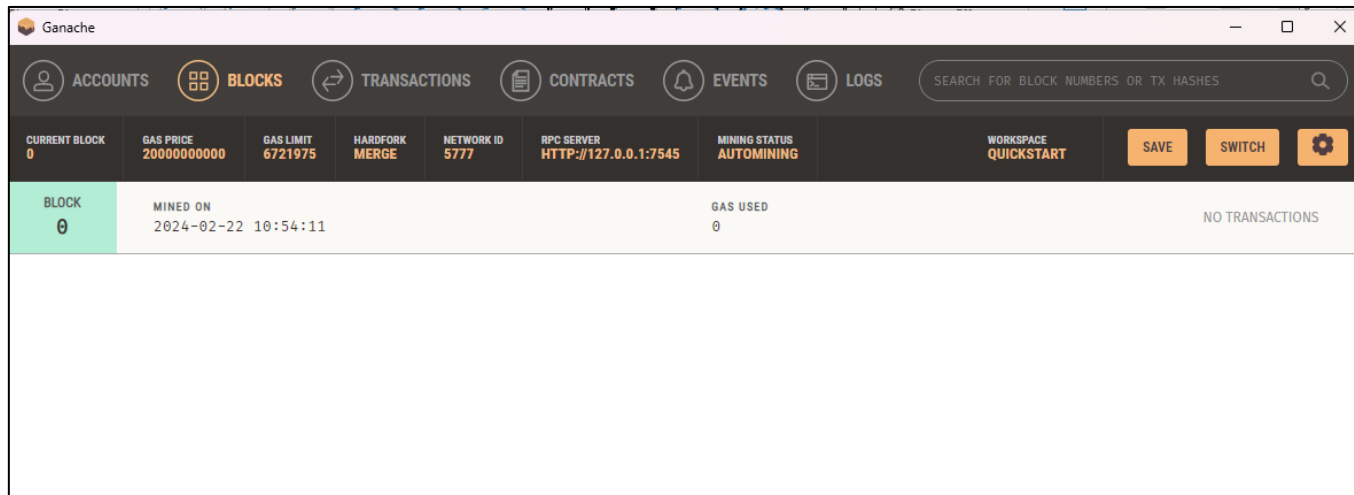
SWITCH

MNEMONIC ?
pilot dismiss nasty improve dish tray hour faint person practice position share

HD PATH
m44'60'0'0account_index

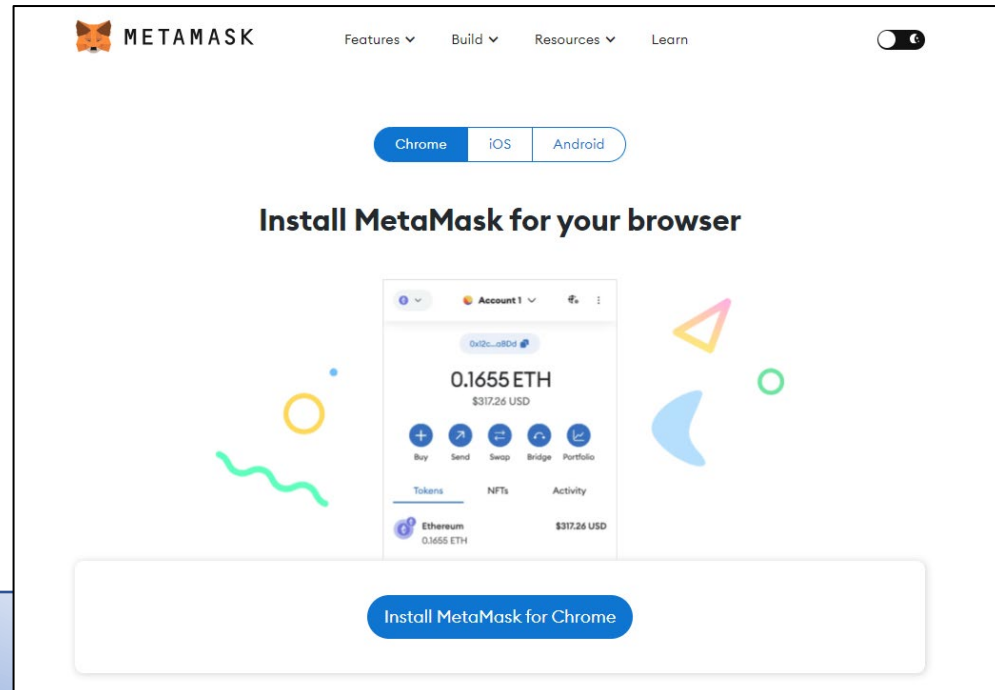
ADDRESS	BALANCE	TX COUNT	INDEX	
0x8CbC90151a916451D32d45fDEbfADC07C24A5952	100.00 ETH	0	0	
0xc71C95Eb05c99c5930af2F1A16242728E0141Fd4	100.00 ETH	0	1	
0xfE02DD214281e5E1f02f503d894949Fe9dE6B93f	100.00 ETH	0	2	
0x04f574c70DcC40989C4469d122C65343CE0Af7dD	100.00 ETH	0	3	
0x3a8dF145A26D589984b73d300a8406EF05E20107	100.00 ETH	0	4	
0x740e4Adb1cccabF84ec4b1172f339c8a0CaE9691	100.00 ETH	0	5	
0x7B0B029553F2d6d9088bc6235B6a61eBa4889Fa4	100.00 ETH	0	6	

Simulating a Blockchain

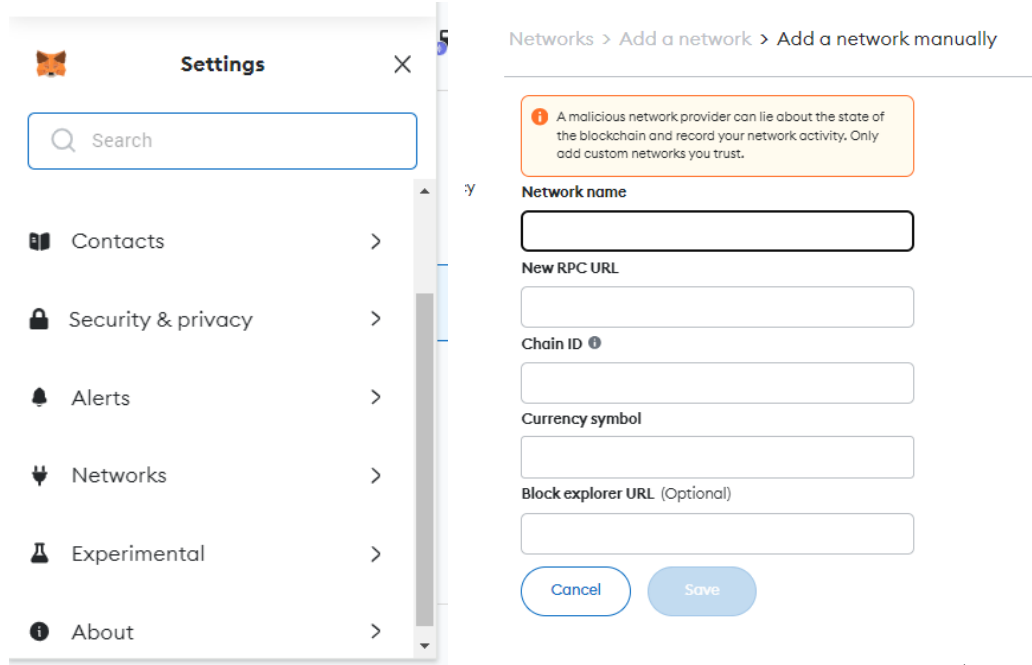


- **Creating a Wallet account**

- Metamask will also allow us to manage our personal account when we connect to the blockchain, as well as manage our Ether funds that we'll need to pay for transactions.
- Link the wallet to the Ethereum account.

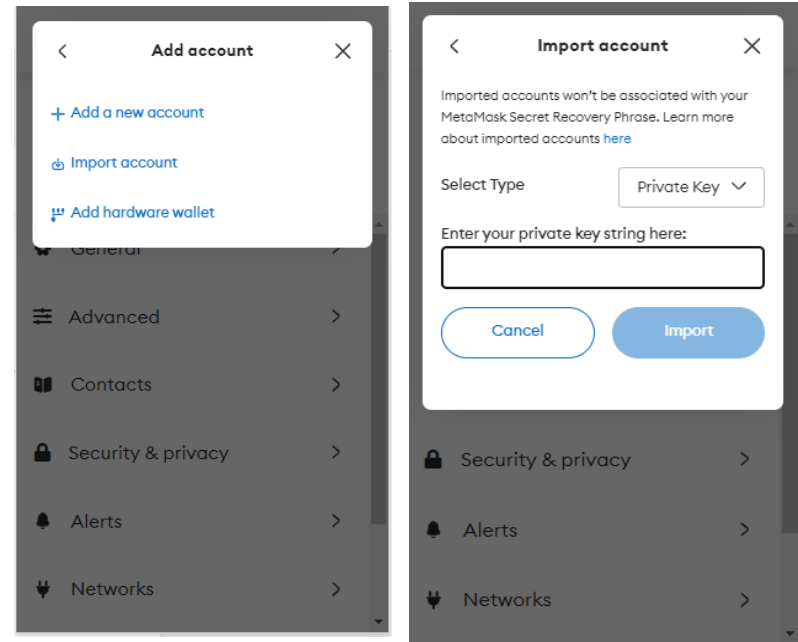


Simulating a Blockchain



The screenshot shows the MetaMask 'Settings' screen. On the left is a sidebar with options: Contacts, Security & privacy, Alerts, Networks, Experimental, and About. The main area is titled 'Networks > Add a network > Add a network manually'. It features a warning box: 'A malicious network provider can lie about the state of the blockchain and record your network activity. Only add custom networks you trust.' Below this are input fields for 'Network name', 'New RPC URL', 'Chain ID', 'Currency symbol', and 'Block explorer URL (Optional)'. At the bottom are 'Cancel' and 'Save' buttons.

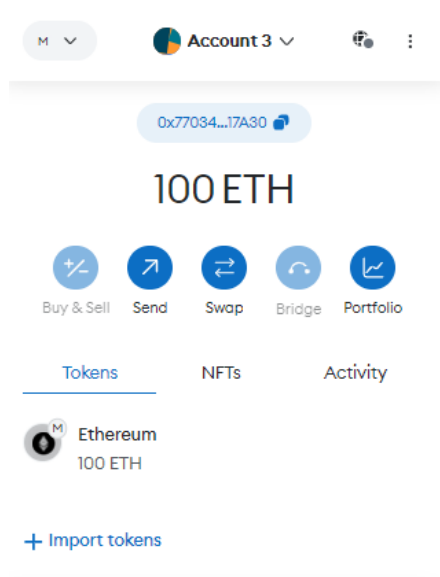
Setup Network



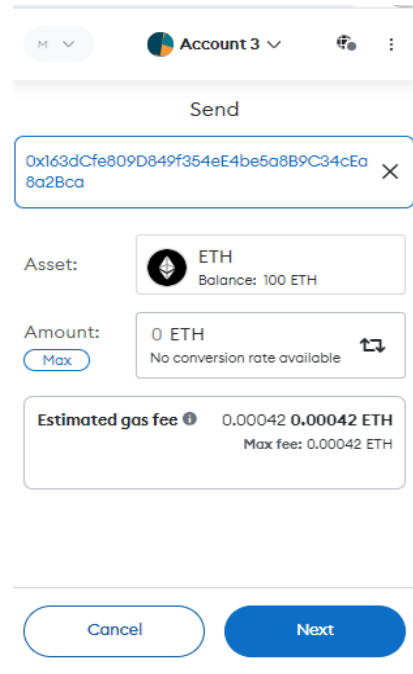
The screenshot shows the 'Add account' screen in MetaMask. It has three options: '+ Add a new account', 'Import account', and 'Add hardware wallet'. The 'Import account' option is selected, leading to the 'Import account' screen. This screen displays a warning: 'Imported accounts won't be associated with your MetaMask Secret Recovery Phrase. Learn more about imported accounts here'. It includes a 'Select Type' dropdown set to 'Private Key', a field to 'Enter your private key string here:', and 'Cancel' and 'Import' buttons.

Link to Ethereum account

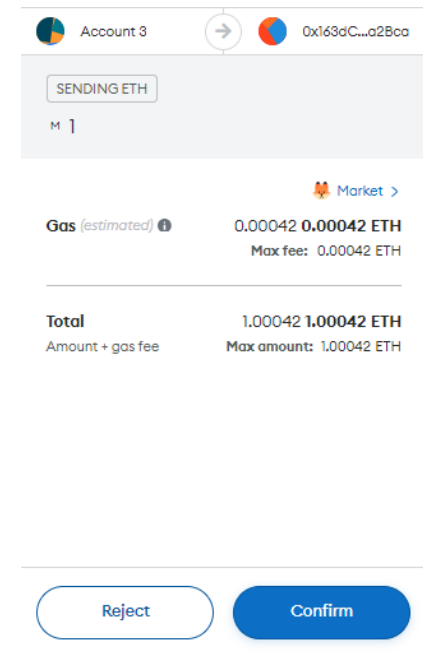
Simulating a Blockchain



Account linked to
wallet



Transaction initiated



Simulating a Blockchain

Account 3

0x77034...17A30

98.9996 ETH

Buy & Sell

Send

Swap

Bridge

Portfolio

Tokens

NFTs

Activity

Feb 22, 2024

Send

Confirmed

-1 ETH

-1 ETH

Transaction done

Ganache

ACCOUNTS BLOCKS TRANSACTIONS CONTRACTS EVENTS LOGS

CURRENT BLOCK 1 GAS PRICE 2000000000 GAS LIMIT 6721975 HARDFORK MERGE NETWORK ID 5777 RPC SERVER HTTP://127.0.0.1:7545 MINING STATUS AUTOMINING WORKSPACE QUICKSTART SAVE SWITCH

MNEMONIC sound skull token copper city smooth erupt slam asthma often income ivory HD PATH m44'60'0"0account_index

ADDRESS	BALANCE	TX COUNT	INDEX	
0x77034732768083f187060260C1Fa4D321a517A30	99.00 ETH	1	0	
ADDRESS	BALANCE	TX COUNT	INDEX	
0x163dCfe809D849f354eE4be5a8B9C34cEa8a2Bca	101.00 ETH	0	1	
ADDRESS	BALANCE	TX COUNT	INDEX	
0x384440bAEA2Cb38d9DBFC351aAF0DF719e83F625	100.00 ETH	0	2	
ADDRESS	BALANCE	TX COUNT	INDEX	
0xF5D27F0ED924cEFAf66B20Ce8a4d3273E36C0846	100.00 ETH	0	3	
ADDRESS	BALANCE	TX COUNT	INDEX	
0xD28479D2F114c499457BEa48F96a10F28Be9825C	100.00 ETH	0	4	
ADDRESS	BALANCE	TX COUNT	INDEX	
0x7F048f8ccb4490AA013906c1AB400ab7A1411BD4	100.00 ETH	0	5	
ADDRESS	BALANCE	TX COUNT	INDEX	
0xa8EBE0784bD45ad753B132bc5814a50de7C0F876	100.00 ETH	0	6	

Simulating a Blockchain

CURRENT BLOCK	GAS PRICE	GAS LIMIT	HARDFORK	NETWORK ID	RPC SERVER	MINING STATUS	WORKSPACE	SAVE	SWITCH	Settings
1	20000000000	6721975	MERGE	5777	HTTP://127.0.0.1:7545	AUTOMINING	QUICKSTART			
BLOCK 1	MINED ON 2024-02-22 13:41:12		GAS USED 21000		1 TRANSACTION					
BLOCK 0	MINED ON 2024-02-22 11:47:15		GAS USED 0		NO TRANSACTIONS					

ACCOUNTS	BLOCKS	TRANSACTIONS	CONTRACTS	EVENTS	LOGS	SEARCH FOR BLOCK NUMBERS OR TX HASHES				
CURRENT BLOCK 1	GAS PRICE 20000000000	GAS LIMIT 6721975	HARDFORK MERGE	NETWORK ID 5777	RPC SERVER HTTP://127.0.0.1:7545	MINING STATUS AUTOMINING	WORKSPACE QUICKSTART	SAVE	SWITCH	Settings
TX HASH 0x7413b623b05d9029ada814ed7f611cd391f31d4ea12a18a28b160c9cbab691c4 VALUE TRANSFER										
FROM ADDRESS 0x77034732768083f187060260C1Fa4D321a517A30			TO ADDRESS 0x163dCfe809D849f354eE4be5a8B9C34cEa8a2Bca			GAS USED 21000		VALUE 10000000000000000000		

Simulating a Blockchain

Fork, modify and/or improve an existing blockchain open-source code

- A **fork** in software development means making a copy of something separate from the original thing.
- Essential for technology adaptation.
- Forking the Ethereum blockchain means copying the Ethereum blockchain's state at a certain block and making a copy of it to make your changes moving forward.
- A blockchain fork is essentially a split of any existing blockchain. Basically, there are two types of forks we can define: soft and hard forks.
 - **Soft forks:**
 - The soft forks are those that are aimed to insert functional changes within the network so that the system will remain the same, yet all current nodes should be updated to the new rules.
 - It is backward compatible, i.e. the nodes with older software are allowed to create new blocks unless they break any protocol rules laid down by the update.
 - **Hard forks:**
 - A hard fork is a permanent divergence made in a Blockchain that no longer accepts the blocks created by nodes that have not upgraded themselves.
 - Adding a new rule in the Blockchain adds a fork in the Blockchain. This creates two new paths: one path that follows the new rules, the upgraded blockchain and an older path, which is yet to upgrade. As more and more nodes realize that their version of blocks is outdated, they would upgrade.
 - A hard fork is incompatible with the previous version of the Blockchain.
- Ganache CLI